Group 29 Progress Report:
IoT Attack Detection Machine Learning Model

Jacob Brodersen, Daniel Maurer, Olivia Reich
{brodersj, maured2, reicho}@mcmaster.ca

1 Introduction

The Internet of Things (IoT) encapsulates a network of numerous interconnected devices and technologies, facilitating real-time communication and data sharing. In this field, a major concern is the identification of malicious network traffic, an issue only growing in scale as time goes on. IoT connected devices are becoming extremely popular with a predicted 29 billion IoT connected devices by 2030 (Katigbak, 2025). However, due to network complexities as well as the diverse technologies and communication channels integrated through the system, there are many possible areas of weakness that leave IoT devices susceptible to cyber-attacks and malicious activity. Almost all IoT systems are at risk of attack, regardless of the active security measures put into place.

As a means of risk mitigation, this project aims to classify network traffic as one of 12 different classes including 9 malicious behaviors and 3 benign behaviors. Such a classifier can be used within Intrusion Detection Systems (IDSs), resulting in more robust and adaptive security for IoT networks (Sharmila and Nagapadma, 2024). Many such IoT network monitoring applications exist, either as software bundled with the purchase of a related IoT device, or as standalone applications. These applications tend to focus on not only security, but other important statistics including device performance, health, and availability (Cooper, 2025). Our solution will focus purely on identifying and intercepting possible attacks. Our work is available on the 4AL3 IoT Threat Detector GitHub repository (Brodersen et al., 2025).

1.1 Related Work

We have identified two main papers that aim to complete the same task as our project. Saba et.al focus on applying Convolutional Neural Networks (CNNs) to anomaly-based intrusion detection systems, arguing that deep learning's pattern matching abilities are superior to the more traditional machine learning techniques used to find anomalies in IoT infrastructure (Saba et al., 2024).

Sharmila et.al, the creators of the dataset we employ, utilize Vector-quantized Variational Autoencoders (VQAEs) as their model of choice. The major argument they make against the above approach is incompatibility with the limited processing power of most IoT devices. They propose QAEs as a superior model, also utilizing an anomaly-based detection approach (Sharmila and Nagapadma, 2023).

Rahman et.al work on a similar problem in the field of IoT – device classification, a task becoming ever-difficult as IoT device popularity grows and an increasing number of devices are created. The group created an extensive new data set and evaluated various models, with Random Forest performing the best (Rahman et al., 2025).

Four other papers that work on similar problems in classifying general internet traffic are discussed as follows. Fesl et.al work on classifying both the protocol and content of VPN traffic, experimenting with Recurrent Neural Networks (RNNs) amongst other methods to classify packets (Fesl and Naas, 2025). They experience some success, but Random Forest still outperforms on their dataset; this may or may not indicate their success with classifying IoT traffic. Latif-Martinez et.al produce a novel approach for anomaly detection in general network monitoring with a modified version of Graph Neural Networks (GNNs), which exhibits considerable performance increases on benchmark data sets (Latif-Martínez et al., 2025). Kim et.al investigate encrypted traffic classification; encryption methods lead to challenges in labelling and imbalanced datasets. They experiment with extracting non-biased features, as well as features

involving packet aggregation to improve identification rates (Kim and Kim, 2024). Lastly, Ness et.al trial various models on benchmark internet package datasets, with gradient boosting techniques exhibiting the highest accuracy (Ness et al., 2025).

The high performance of models listed in the above papers, aside from our original standard neural network experimented with in our initial report, confirmed our confidence in other models that we investigate within this report. Namely, we trial decision trees and support vector classifiers on the dataset, less complex models that we use as comparison against the neural network. With these experiments we hope to challenge our early assumption that only sufficiently complex networks can perform well on this dataset.

2 Dataset

2.1 Original Dataset

The original dataset can be found on the UC Irvine Machine Learning Repository (Sharmila and Nagapadma, 2024) and is also available on our GitHub repository. This dataset has 123,117 instances and includes 84 input features and one target. The dataset includes two categorical input features, proto and service. Proto has three categories. Service has nine categories (and one category of empty). The target, Attack_type, is also categorical. Attack_type has 12 categories with three being benign network traffic and nine being malicious network traffic. Note that one category of the Attack_type, dos_syn_hping, represents 76.9% of the dataset. On the other hand, the Attack_types, Metasploit_Brute_Force_SSH and NMAP_FIN_SCAN, make up less than 0.1% of the dataset. Finally, one of the input features, bwd_URG_flag_count, is zero for every instance in the dataset. The other 82 input features are numerical.

2.2 Preprocessing Implementation

For preprocessing, we perform three main transformations. First, we remove the features: id, id.orig_p, and id.resp_p. As discussed in Section 3, these features do not actually have predictive value towards benign or malicious attack behavior and should be removed. Additionally, we also removed bwd_URG_flag_count as it is zero for every instance and therefore does not have predictive value. Second, we encode the categorical features to numerical values. This is done using pandas's get_dummies function. This function takes a categorical feature and splits it into a one-hot vector of the same length as the number of categories that feature has. This was chosen instead of simply mapping a number to each class to avoid implicitly defining an order over the categories. Third, we perform z-score normalization on the input data so that all features are uniformly scaled.

2.3 Preprocessed Dataset

After preprocessing, the input data has 91 features. The increased number of features is a result of transferring the two categorical features into one-hot vectors. In the preprocessed dataset, all the features are numeric and normalized. The target data now has 12 numeric features. Again, there are 12 features because each one corresponds to a category for the Attack_type one-hot vector.

This preprocessed version of the input and target can be created from the original dataset using the preprocess.py script.

2.4 Post-Progress Report Changes

There is one change we made to the dataset between the progress report and this final version. As mentioned above, the dos_syn_hping target class represents 76.9% of the dataset. This creates bias in the dataset, and we believe that this is a factor in making the dataset too easy. We tried to address this bias in a few different ways. First, we first tried downsampling by making all classes have the same number of instances (using the target class with the smallest number of instances). Unfortunately, by doing this, there are not enough instances in the dataset as required by the project (328<1000) as well as not a meaningful number of training instances to properly train the model. Because of this, instead of removing instances from other target classes, we tried randomly sampling 10,000 instances from the dos_syn_hping target class. This preserves its place as the largest target class represented, but now only takes up 26% of the dataset. The final approach we used was upsampling the training data. This was done by keeping the most represented class in the training set the same, but then randomly duplicating instances of the less represented classes until all target classes are equally represented in the training data.

3 Features

The majority of the features for our dataset were created using the Zeek network monitoring tool and a plugin called Flowmeter. The description of these features, from (Gambazzi, 2025), can be found in the appendix in Table 6. Features not described by (Gambazzi, 2025) are:

- id: Instance number for specific Attack_type class
- id.orig_p: This is the port number of origin device.
- id.resp_p: This is the port number of response device.
- proto: This is the transport layer protocol of the communication.
- service: This is the application layer protocol of the communication.

No feature engineering was done as all the features were already given in the dataset.

For feature selection, id, id.orig_p, and id.resp_p are not indicators of benign/malicious network activity. Additionally, from profiling the dataset, we know that the bwd_URG_flag_count feature is a constant value across all instances. For the above reasons, we should not include id, id.orig_p, id.resp_p, or bwd_URG_flag_count in our feature set. We have tried both keeping all other features, as well as removing features that have high (absolute) correlation with each other. In the latter case, we removed features (one in a pair) with an absolute correlation equal to or above 0.9. This resulted in the removal of 29 features:

- fwd_pkts_tot
- bwd_data_pkts_tot
- fwd_pkts_per_sec
- flow_pkts_per_sec
- fwd_header_size_min
- bwd_header_size_tot
- bwd_header_size_min
- flow_ACK_flag_count
- fwd_pkts_payload.std
- bwd_pkts_payload.tot
- bwd_pkts_payload.std
- flow_pkts_payload.max
- flow_pkts_payload.tot
- fwd_iat.max
- fwd_iat.tot
- fwd_iat.std
- bwd_iat.std
- flow_iat.tot
- flow_iat.std
- payload_bytes_per_second
- bwd_subflow_bytes
- fwd_bulk_packets
- bwd_bulk_packets
- active.max
- idle.min
- idle.max
- idle.tot
- idle.avg
- proto_udp

The only feature augmentation we performed was normalizing the data as described in Section 2.2.

4 Implementation

Our first model implementation is a standard neural network with no recurrent or convolutional layers. Defined using PyTorch, the model was designed to be as general as possible to facilitate the tuning of hyperparameters beyond learning rate or regularization strength, as we also wish to experiment with network width and layer depth. To create a model instance, the input dimension (e.g. the number of features), output dimension (e.g. number of classes), and dimensions of all hidden layers within a list are taken as inputs. Note that the number of layers is also fully customizable, reflecting how long the list is. A sequential neural network is then created with these parameters, with the activation function between each layer chosen as ReLU due to its simplicity and efficiency. The raw output of the model, given a singular instance's features, is a vector with length reflecting the number of classes, with the most positive argument being the model's prediction.

The chosen loss function for the model is Cross

Entropy loss, selected due to its ability to penalize incorrect predictions stronger in comparison to a loss function such as MSE. This loss is evaluated on each iteration's batch, with this loss used to calculate the gradient that updates the model's parameters. The full cross-entropy loss across all instances is also calculated every few iterations but does not play a role in updating parameters and is used to track model training progress. Within PyTorch, this function implicitly calculates the softmax on input vectors, hence why this layer is left out of the model. With similar reasoning, to compare model outputs to target labels when calculating the F1-score or accuracies, both the model output and target label vectors are run through the argmax function to convert to a single integer reflecting the classes. Similarly, the F1-score is also calculated every few iterations, but its value does not influence updates to model parameters.

To optimize, we implemented mini-batch stochastic gradient descent, both to reduce the computational load of training the model on the 120,000 instances of the original dataset at once and to take advantage of its ability to escape local minimums. This process is not changed after downsampling of the dataset to continue to take advantage of these benefits. Batch size is also configurable, allowing it to be tuned as a hyperparameter. To ensure randomness in batch selection, the training data is shuffled once per epoch, ensuring that possible bias from ordering of instances is reduced. The Adam optimizer is also implemented in order to take advantage of its improvements over SGD, which include momentum and adaptive learning rates. Currently, no methods of regularization (L1, L2, or dropout) are implemented as the model does not appear to rapidly overfit, however the model has been designed such that this functionality can be easily implemented in future iterations.

In total, the list of model hyperparameters available to be experimented with are as follows: learning rate, number of layers, depth of each layer, batch size, and number of iterations. Within future development of this project, we may look into implementing additional forms of hyperparameter tuning with both methods of regularization and early stopping. If that is the case, regularization strength, dropout probability, and patience will be additional hyperparameters. For each specified hyperparameters, a specific range of hyperparameter values was chosen as possible options. The itertools import from Python was then used to create a Cartesian product that contained all possible combinations of the possible hyperparameter values. A random sample of 50 combinations was then chosen in order to conduct random search. Due to the large amount of data and high computational demands of training the model, random search was chosen over grid search to avoid excessively long and demanding computations during hyperparameter tuning.

There are two other models that we also experiment with and use as alternative comparisons as relatively simpler alternatives to our neural network. The first is a decision tree, implemented with sklearn. No changes were made to the default settings to encourage simplicity of the model. The second is a support vector classifier with an RBF kernel, also implemented via sklearn, utilized as a slightly more complex alternative to the network-based baseline to follow. Our primary comparison is a simple logistic regression model, with an identical loss function and optimization method as described above. Training in a similar way is done to establish the baseline as a less complex version of our neural network. Outperforming this baseline would demonstrate that the added complexity of multiple non-linear layers is necessary for accurately solving the problem.

Our final baseline is a set of results found on Kaggle (Shakeel, 2025), which act as related work we aim to match performance with. On tested models such as the decision tree and neural network, accuracy of 0.9943 and 0.9890 respectively were achieved. Direct comparison with this work across all experiments is not fully possible as this work does not downsample the dataset, remove correlated features, or upsample as we do within this report, however these results can still indicate whether our models perform as expected, and act as a general measuring stick for our experiments. As demonstrated within the results to follow, we match this baseline.

5 Results and Evaluation

5.1 Method of Evaluation

Currently, the data is partitioned such that approximately 80% is used for training, 10% for hyperparameter tuning via validation, and the remaining 10% is reserved as a held-out test set. This test set is not part of the training and instead is used to simulate our model's effectiveness on real-world data,

as this data has not been exposed during the training process and only tests on our complete model after hyperparameter tuning. Due to the large amount of data within this dataset, the need for k-fold validation wasn't as imperative, as 98,000 data points proved to be enough while executing model training. Similarly, ~12,000 - 13,000 data points are sufficient for both hyperparameter tuning as well as verification that the model executes to an acceptable degree on real-world data.

During hyperparameter tuning, there were 3 batch options, 4 learning rate options, 4 iteration count options and 20 hidden structure options chosen at random, each containing 2–4 layers, with layer sizes selected from six possible depth configurations. The best results from all options within batch size, learning rate and iteration count were showcased on a graph. Due to the expansive list of possible hidden structures, it is not evaluated on its own graph but simply noted as one of the parameters used to generate the best results in the other hyperparameter graphs in the legend.

During the progress report, the only metrics used for evaluation were F-scores and loss. These remain our primary metrics for assessing the progress of our neural network model. This means that they are the only metrics we track throughout the training iteration of our neural network model. These metrics proved sufficient to be able to assess whether the model shows meaningful trends throughout the learning process. However, we expanded metrics for the final evaluation. In addition to F-score and loss, the following metrics were added for final evaluation: overall accuracy, accuracy (per class), precision (per class), and recall (per class). Due to the high results achieved from our model, these metrics were added to provide a more fine-grained understanding of where our model could be improved. The metrics that evaluate per-class data are particularly important additions in this regard because they help break down where failure in the model may be occurring. This is especially true if it can be linked back to high failure within specific classes.

5.2 Results

Overall results are summarized in Table 1. Further detailed results regarding per-class performance across the different models have been included in the Appendix for reference and can be summarized within the following tables: Table 2, Table 3, Table 4, Table 5

Details regarding the models' parameters used to derive these results have been included below.

Parameters for logistic regression basline: optimizer=adam, batch_size=64, iterations=10000, learning_rate=0.001,

Parameters for decision tree baseline: kernel=rbf, C=1.0, gamma=scale

Parameters for SCV: criterion=gini, splitter=best, default max depth

Parameters for neural network (as determined through hyperparameter tuning): batch_size=128, architecture=[128, 128], learning_rate=0.0001, iterations=50000

6 Progress

Following our progress report, our plan was to implement various other models to see how they compared to our neural network model. The models we intended to implement were a CNN, an RNN, a Decision Tree, and a Support Vector Machine. We have implemented two of these models: Decision Tree and Support Vector Machine. We decided not to implement the CNN or RNN because, after we learned more about them, we realized that they were not appropriate for our data and task. In particular, we originally thought an RNN might be useful due to its effectiveness with time-series data. This would have been appropriate had the data included time-stamped entries, but unfortunately each instance does not have individual packet information. In terms of the CNN, it also does not really make sense as the nature of our data does not follow a grid-like topology.

A second part of our plan was related to the feature selection process. We planned to see how removing highly correlated features affected the performance of our model. We have implemented this as described in the 3 section. We also planned to try selecting features by using the top performing linear regressions as the feature set (where the exact number of features to include can be determined as a hyperparameter). However, we decided not to do this as the correlation feature selection did not really affect performance (described in the 5.2 section). Note that this aligns with TA feedback we received in which we were told that Neural Networks can often handle features themselves.

A third part of the plan was related to removing instances from the dataset in order to have a more equal distribution of target classes. Our original dataset had a large bias. Namely, one target class made up 76.9% of the dataset, and as such, if the model was in doubt, it could choose this class and be right much more frequently than if the classes distribution was more equal. We tried a few different methods to address this bias; these methods are described in the 2.4 subsection.

The final part of the plan was adding another baseline that predicts a specific class every time. Our main interest in doing this was because of the large bias in the dataset. The idea was that this new baseline could predict better than it otherwise would with a dataset that has equally represented classes. Now that the dataset has been augmented to reduce the bias, this baseline is no longer relevant and as a result, we have decided not to include it.

7 Error Analysis

Team Contributions

7.1 Jacob Brodersen

Key Contribution Area: Preprocessing and Dataset Profiling

Contribution Summary: In terms of the code, I worked on the preprocessing, profiling of the dataset, and workflow / script layout. For the report, I worked on Dataset, Features, and Feedback and Plans.

7.2 Daniel Maurer

Key Contribution Area: Model & Training Loop Definition

Contribution Summary: For the code, I created the model definition, as well as the training loop. For the report, I wrote the Model Implementation, Introduction, and Related Work.

7.3 Olivia Reich

Key Contribution Area: Model & Training Loop Definition

Contribution Summary: For the code, I implemented training with hyperparameter tuning, testing and graph visualization . For the report, I wrote a paragraph in the Model Implementation regarding justification for the chosen hyperparameter tuning method of random search as well as Evaluation & Results, the Appendix, and LaTeX formatting.

References

J. Brodersen, O. Reich, and D. Maurer. 2025. 4al3 iot threat detector. Accessed 11 October 2025.

S. Cooper. 2025. The best iot device monitoring tools. Accessed 7 October 2025.

J. Fesl and M. Naas. 2025. A comprehensive machine learning-based approach for virtual private network traffic detection, classification and hiding. *Computer Networks*, 270.

L. Gambazzi. 2025. Zeek flowmeter. Accessed 11 October 2025.

R. Katigbak. 2025. The economy of things: The next value lever for telcos. Accessed 7 October 2025.

Min-Gyu Kim and Hwankuk Kim. 2024. Anomaly detection in imbalanced encrypted traffic with few packet metadata-based feature extraction. *CMES - Computer Modeling in Engineering and Sciences*, 141(1):585–607.

H. Latif-Martínez, J. Suárez-Varela, A. Cabellos-Aparicio, and P. Barlet-Ros. 2025. Gat-ad: Graph attention networks for contextual anomaly detection in network monitoring. *Computers & Industrial Engineering*, 200.

Stephanie Ness, Vishwanath Eswarakrishnan, Harish Sridharan, Varun Shinde, Naga Venkata Prasad Janapareddy, and Vineet Dhanawat. 2025. Anomaly detection in network traffic using advanced machine learning techniques. *IEEE Access*, 13:16133–16149.

M. M. Rahman, F. Bouhafs, S. A. Hoseini, and F. den Hartog. 2025. Unsw homenet: A network traffic flow dataset for ai-based smart home device classification. *Computers & Industrial Engineering*, 204.

T. Saba, A. Rehman, T. Sadad, H. Kolivand, and S. A. Bahaj. 2024. Anomaly-based intrusion detection system for iot networks through deep learning model. *Journal of Network and Computer Applications*.

M. Shakeel. 2025. Rt$_i ot(cps)$. Accessed 2 December 2025.

B. Sharmila and R. Nagapadma. 2023. Quantized autoencoder (qae) intrusion detection system for anomaly detection in resource-constrained iot devices using rt-iot2022 dataset. *Cybersecurity*, 6:1–15.

B. S. Sharmila and R. Nagapadma. 2024. Rt-iot2022 dataset. Accessed 27 September 2025.

Classifier	ID	Parameter	F-score	Accuracy
Logistic Regression	C1	dataset with no modifications	0.9073	0.9790
	C2	removal of features with correlation >= 90%	0.8900	0.9791
	C3	10000 instances of downsampling	0.8627	0.9327
	C4	10000 instances of upsampling	0.9472	0.9470
Decision Tree	C5	dataset with no modifications	0.9283	0.9963
	C6	removal of features with correlation >= 90%	0.9268	0.9962
	C7	10000 instances of downsampling	0.8838	0.9878
	C8	10000 instances of upsampling	0.9976	0.9976
SCV	C9	dataset with no modifications	0.9296	0.9932
	C10	removal of features with correlation >= 90%	0.9196	0.9932
	C11	10000 instances of downsampling	0.8816	0.9756
	C12	10000 instances of upsampling	0.9718	0.9716
Neural Network	C13	dataset with no modifications	0.9518	0.9957
	C14	removal of features with correlation >= 90%	0.9354	0.9354
	C15	10000 instances of downsampling	0.8963	0.9862
	C16	10000 instances of upsampling	0.9887	0.9888

Table 1: Classifier performance on the test set

8 Appendix

ID	category	1	2	3	4	5	6	7	8	9	10	11	12
	accuracy	0.82	0.84	1.00	1.00	0.67	0.75	1.00	0.98	0.97	1.00	0.90	0.53
C1	precision	0.88	0.98	1.00	0.99	1.00	1.00	1.00	1.00	0.95	1.00	0.83	0.89
	recall	0.82	0.84	1.00	1.00	0.67	0.75	1.00	0.98	0.97	1.00	0.90	0.53
	accuracy	0.81	0.82	1.00	1.00	1.00	0.00	1.00	1.00	0.99	1.00	0.89	0.70
C2	precision	0.86	0.96	1.00	1.00	1.00	0.00	1.00	1.00	0.96	1.00	0.83	0.93
	recall	0.81	0.82	1.00	1.00	1.00	0.00	1.00	1.00	0.99	1.00	0.89	0.70
	accuracy	0.82	0.84	1.00	1.00	0.67	0.75	1.00	0.98	0.97	1.00	0.90	0.50
C3	precision	0.88	0.95	1.00	1.00	1.00	0.60	0.99	1.00	0.96	1.00	0.83	1.00
	recall	0.82	0.84	1.00	1.00	0.67	0.75	1.00	0.98	0.97	1.00	0.90	0.50
	accuracy	0.76	0.99	1.00	1.00	0.88	0.90	1.00	1.00	0.92	1.00	0.94	0.98
C4	precision	0.75	0.94	1.00	1.00	0.97	0.97	1.00	0.99	0.99	1.00	0.82	0.95
	recall	0.76	0.99	1.00	1.00	0.88	0.90	1.00	1.00	0.92	1.00	0.94	0.98

Table 2: Per-class performance for Logistic Regression (rounded to 2 decimals)

ID	category	1	2	3	4	5	6	7	8	9	10	11	12
	accuracy	0.99	0.89	1.00	1.00	0.67	0.75	1.00	1.00	0.97	1.00	0.98	0.97
C5	precision	0.98	0.97	1.00	1.00	0.67	0.60	1.00	1.00	0.97	1.00	0.98	0.94
	recall	0.99	0.89	1.00	1.00	0.67	0.75	1.00	1.00	0.97	1.00	0.98	0.97
	accuracy	0.99	0.87	1.00	1.00	1.00	0.00	1.00	1.00	0.99	1.00	0.97	0.90
C6	precision	0.97	0.96	1.00	1.00	0.75	0.00	1.00	1.00	0.96	1.00	0.99	0.90
	recall	0.99	0.87	1.00	1.00	1.00	0.00	1.00	1.00	0.99	1.00	0.97	0.90
	accuracy	0.99	0.90	1.00	1.00	0.67	0.75	1.00	0.99	0.96	1.00	0.98	0.97
C7	precision	0.98	0.95	1.00	1.00	0.67	0.60	1.00	0.99	0.96	0.99	0.99	0.94
	recall	0.99	0.90	1.00	1.00	0.67	0.75	1.00	0.99	0.96	1.00	0.98	0.97
	accuracy	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.98	1.00
C8	precision	0.98	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
	recall	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.98	1.00

Table 3: Per-class performance for Decision Tree (rounded to 2 decimals)

ID	category	1	2	3	4	5	6	7	8	9	10	11	12
C9	accuracy	0.99	0.84	1.00	0.99	0.67	0.75	1.00	0.98	0.97	1.00	0.95	0.57
	precision	0.94	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.96	1.00	0.98	0.94
	recall	0.99	0.84	1.00	0.99	0.67	0.75	1.00	0.98	0.97	1.00	0.95	0.57
C10	accuracy	0.98	0.82	1.00	1.00	1.00	0.00	1.00	1.00	0.99	1.00	0.93	0.70
	precision	0.92	1.00	1.00	1.00	1.00	0.00	1.00	1.00	0.96	1.00	0.98	0.93
	recall	0.98	0.82	1.00	1.00	1.00	0.00	1.00	1.00	0.99	1.00	0.93	0.70
C11	accuracy	0.99	0.84	1.00	0.99	0.67	0.75	1.00	0.98	0.97	1.00	0.95	0.57
	precision	0.94	1.00	1.00	1.00	1.00	0.75	1.00	1.00	0.96	1.00	0.98	0.89
	recall	0.99	0.84	1.00	0.99	0.67	0.75	1.00	0.98	0.97	1.00	0.95	0.57
C12	accuracy	0.97	0.99	1.00	1.00	0.88	0.97	1.00	1.00	0.92	1.00	0.95	1.00
	precision	0.82	0.94	1.00	1.00	0.98	0.98	1.00	1.00	1.00	1.00	0.98	0.99
	recall	0.97	0.99	1.00	1.00	0.88	0.97	1.00	1.00	0.92	1.00	0.95	1.00

Table 4: Per-class performance for SVC (rounded to 2 decimals)

ID	category	1	2	3	4	5	6	7	8	9	10	11	12
C13	accuracy	1.00	0.84	1.00	1.00	0.67	0.75	1.00	1.00	0.97	1.00	0.97	0.90
	precision	0.96	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.96	1.00	0.99	0.96
	recall	1.00	0.84	1.00	1.00	0.67	0.75	1.00	1.00	0.97	1.00	0.97	0.90
C14	accuracy	0.98	0.82	1.00	1.00	1.00	0.00	1.00	1.00	1.00	1.00	0.97	0.95
	precision	0.97	1.00	1.00	1.00	1.00	0.00	1.00	1.00	0.96	1.00	0.98	0.90
	recall	0.98	0.82	1.00	1.00	1.00	0.00	1.00	1.00	1.00	1.00	0.97	0.95
C15	accuracy	0.99	0.84	1.00	1.00	0.67	0.75	1.00	0.99	0.97	1.00	0.97	0.67
	precision	0.96	1.00	1.00	1.00	1.00	1.00	1.00	0.99	0.96	1.00	0.98	0.87
	recall	0.99	0.84	1.00	1.00	0.67	0.75	1.00	0.99	0.97	1.00	0.97	0.67
C16	accuracy	0.97	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.93	1.00	0.97	1.00
	precision	0.97	0.94	1.00	1.00	0.99	0.99	1.00	1.00	0.99	1.00	0.98	1.00
	recall	0.97	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.93	1.00	0.97	1.00

Table 5: Per-class performance for Neural Network (rounded to 2 decimals)

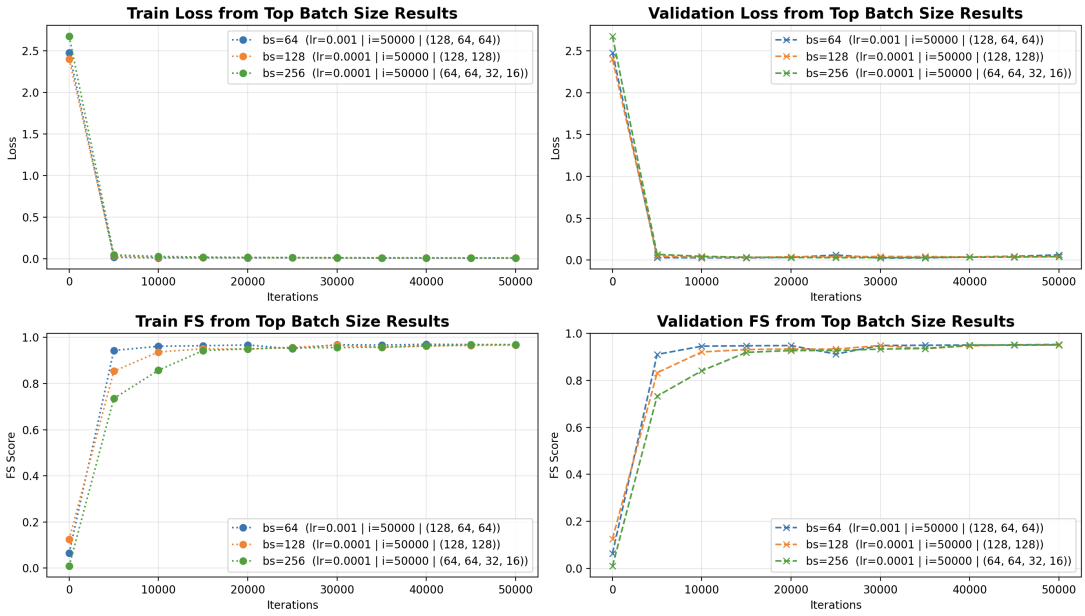

Figure 1: Results for the top-performing model configurations across batch sizes. All data given was used. (a) Training Loss (top-left). (b) Validation Loss (top-right). (c) Training F-score (bottom-left). (d) Validation F-score (bottom-right).

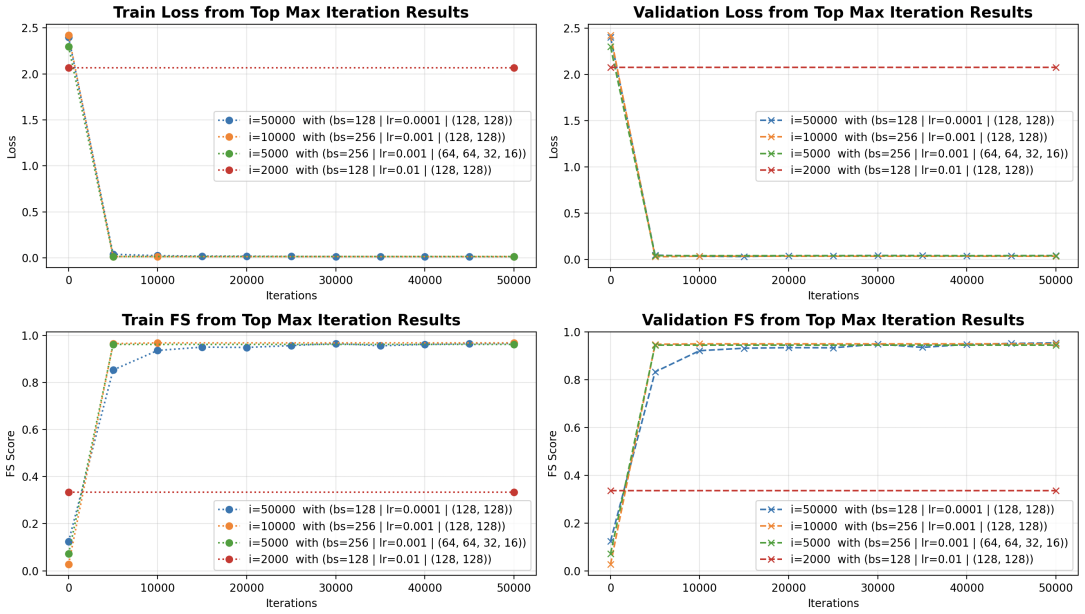

Figure 2: Results for the top-performing model configurations across learning rates (1e-1, 5e-2, 1e-2, 5e-3). All data given was used. (a) Training Loss (top-left). (b) Validation Loss (top-right). (c) Training F-score (bottom-left). (d) Validation F-score (bottom-right).

Figure 3: Results for the top-performing model configurations across maximum iteration counts (2000, 5000, 10000, 50000). All data given was used. (a) Training Loss (top-left). (b) Validation Loss (top-right). (c) Training F-score (bottom-left). (d) Validation F-score (bottom-right).

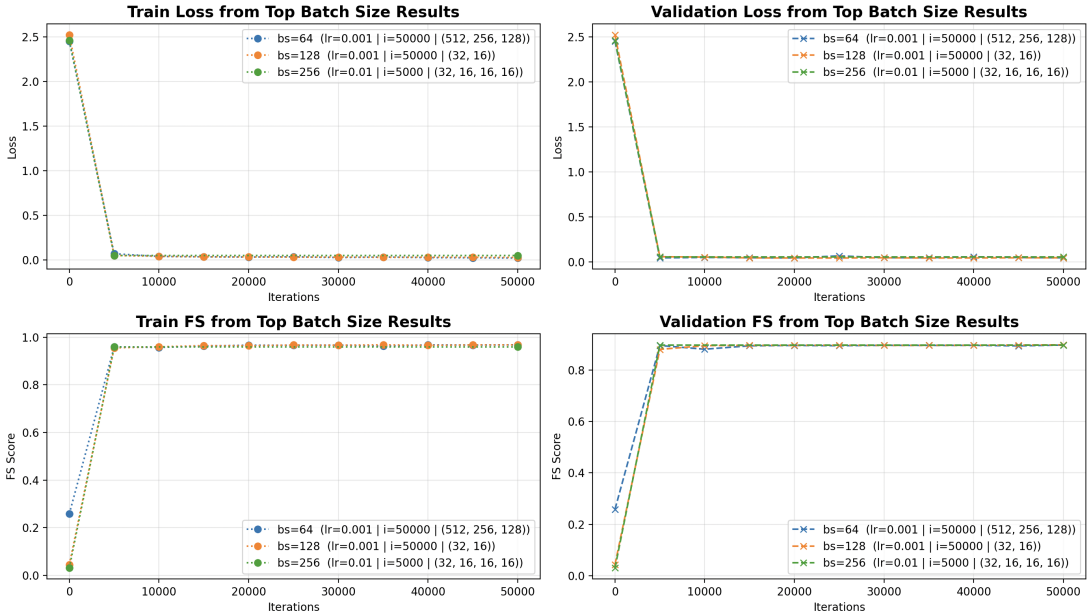

Figure 4: Results for the top-performing model configurations across batch sizes. Features with over 90% correlation were removed from the dataset. (a) Training Loss (top-left). (b) Validation Loss (top-right). (c) Training F-score (bottom-left). (d) Validation F-score (bottom-right).

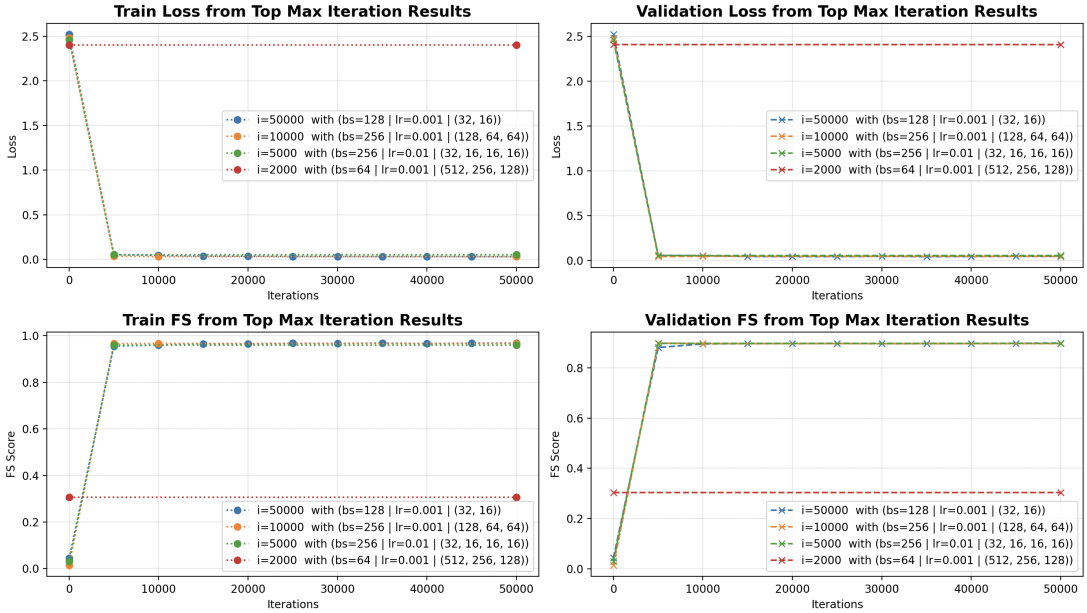

Figure 5: Results for the top-performing model configurations across learning rates where dataset remained unmodified. (1e-1, 5e-2, 1e-2, 5e-3). Features with over 90% correlation were removed from the dataset. (a) Training Loss (top-left). (b) Validation Loss (top-right). (c) Training F-score (bottom-left). (d) Validation F-score (bottom-right).

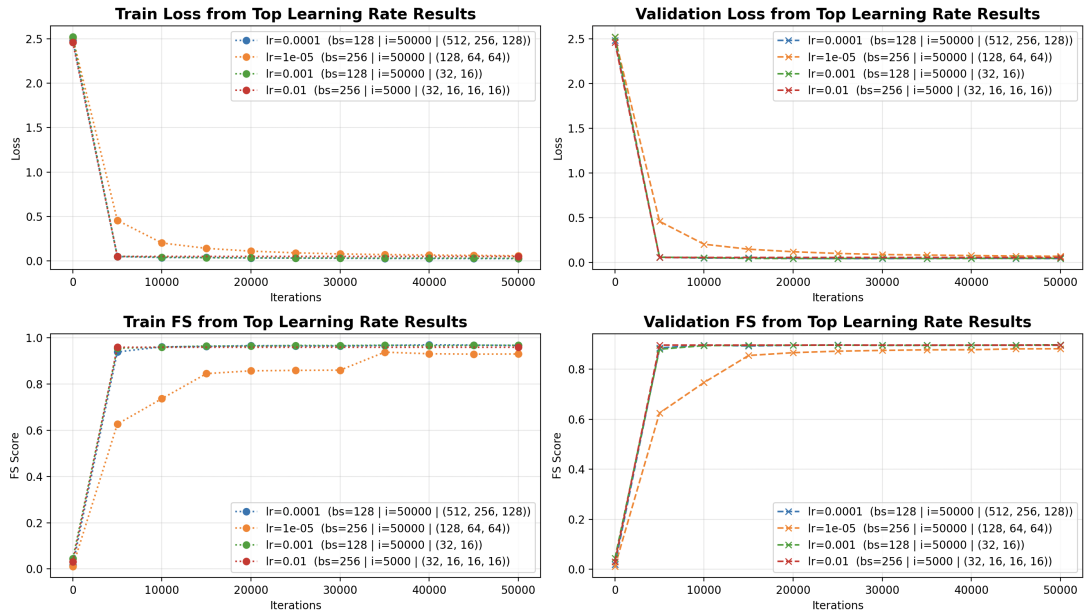

Figure 6: Results for the top-performing model configurations across maximum iteration counts (2000, 5000, 10000, 50000). Features with over 90% correlation were removed from the dataset. (a) Training Loss (top-left). (b) Validation Loss (top-right). (c) Training F-score (bottom-left). (d) Validation F-score (bottom-right).

Figure 7: Results for the top-performing model configurations across batch sizes. Data was reduced to 10,000 instances from downsampling. (a) Training Loss (top-left). (b) Validation Loss (top-right). (c) Training F-score (bottom-left). (d) Validation F-score (bottom-right).

Figure 9: Results for the top-performing model configurations across maximum iteration counts (2000, 5000, 10000, 50000). Data was reduced to 10,000 instances from downsampling. (a) Training Loss (top-left). (b) Validation Loss (top-right). (c) Training F-score (bottom-left). (d) Validation F-score (bottom-right).

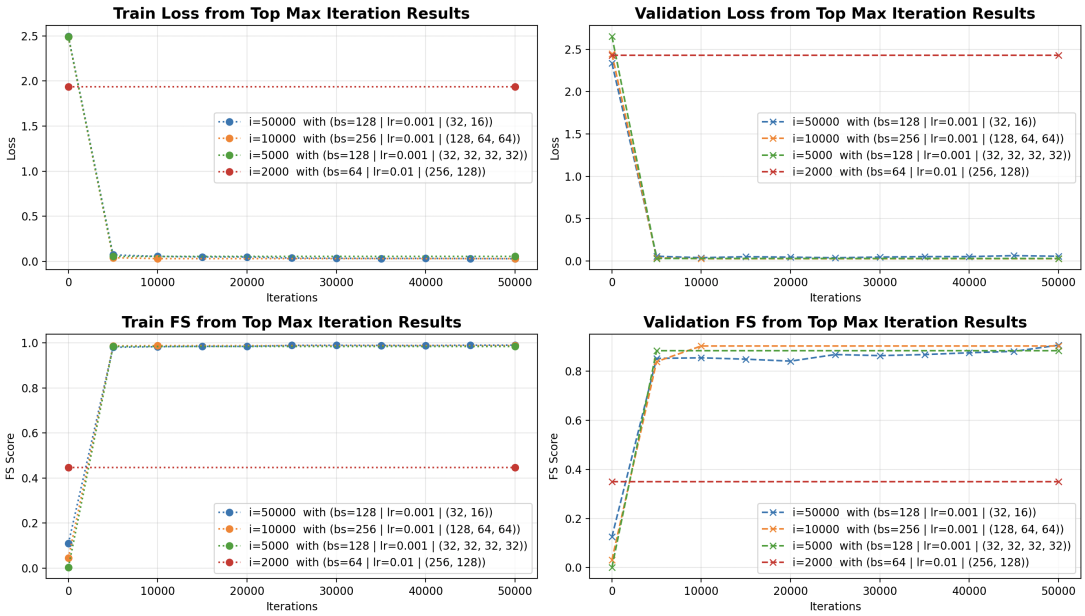

Figure 10: Results for the top-performing model configurations across batch sizes. Data was reduced to 10,000 instances Data was reduced to 10,000 instances from upsampling. (a) Training Loss (top-left). (b) Validation Loss (top-right). (c) Training F-score (bottom-left). (d) Validation F-score (bottom-right).

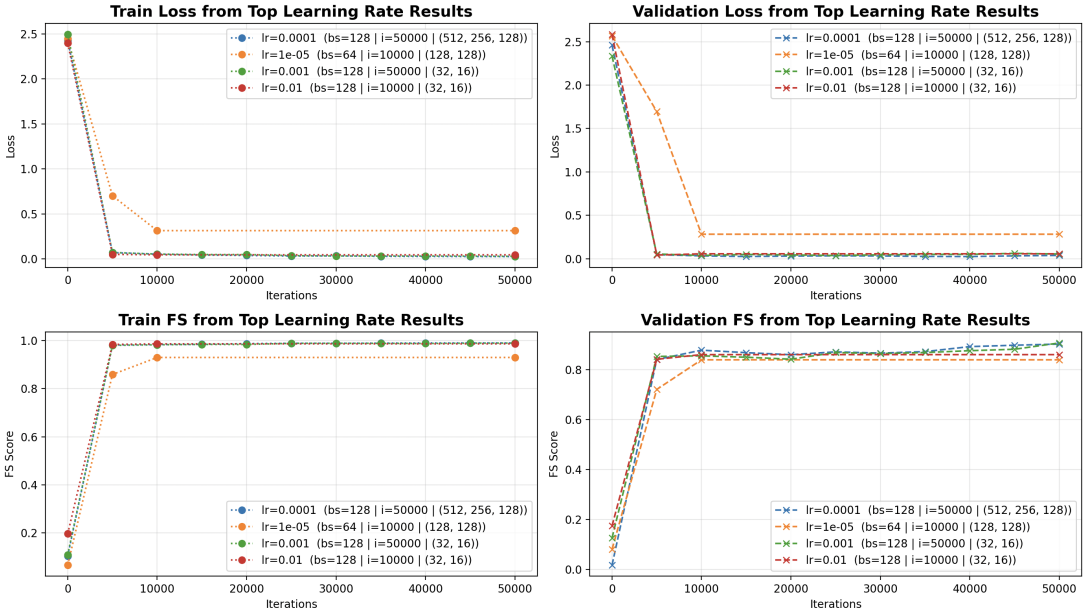

Figure 12: Results for the top-performing model configurations across maximum iteration counts (2000, 5000, 10000, 50000). Data was reduced to 10,000 instances from upsampling. (a) Training Loss (top-left). (b) Validation Loss (top-right). (c) Training F-score (bottom-left). (d) Validation F-score (bottom-right).

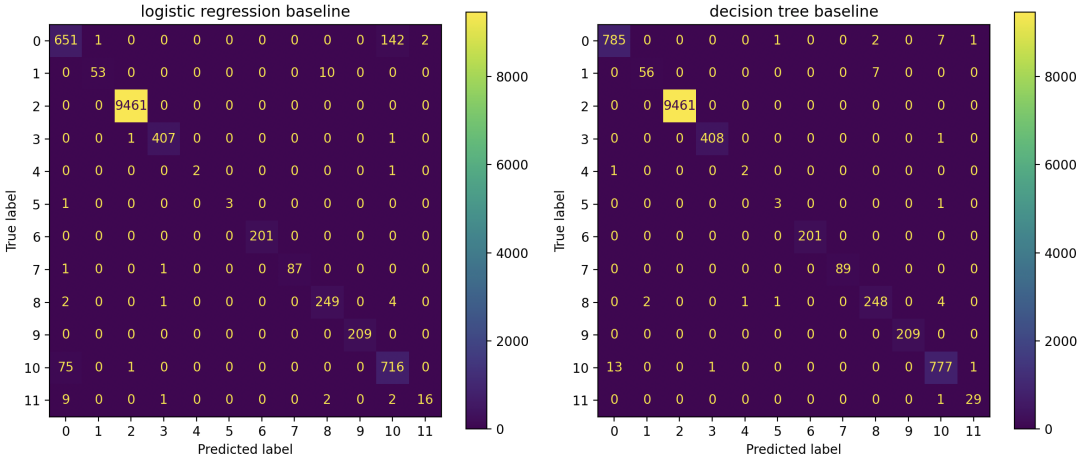

Figure 13: Confusion matrices for the logistic regression (left) and decision tree (right) baselines when all data given was used..

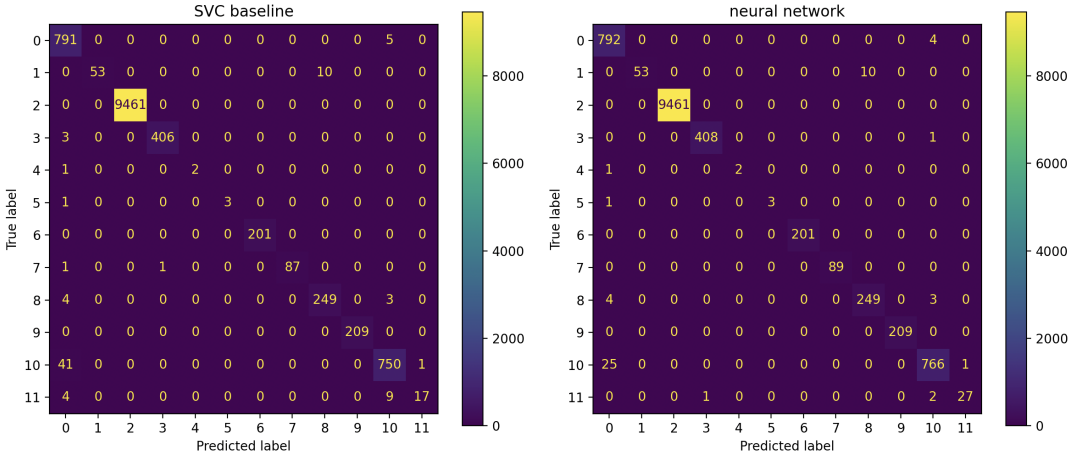

Figure 14: Confusion matrices for the SVC (left) and neural network (right) baselines when features with over 90% correlation were removed from the dataset.

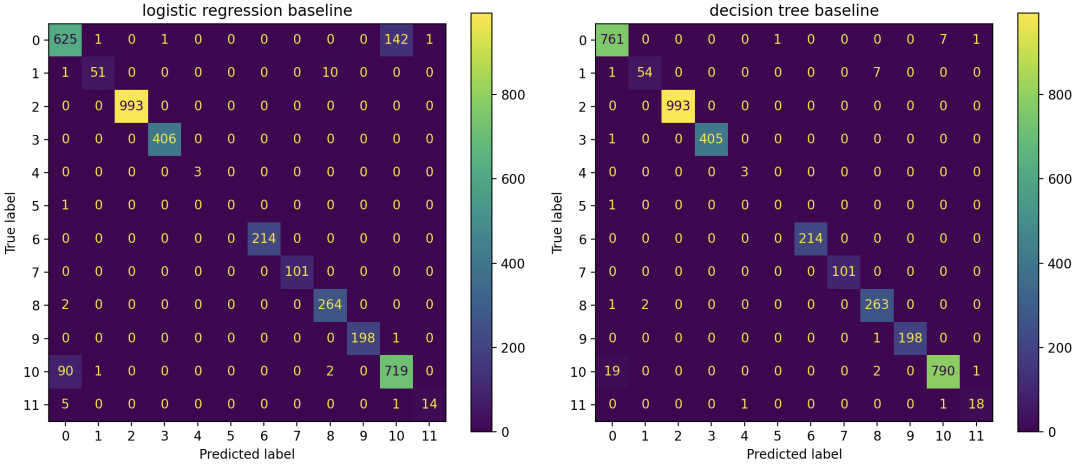

Figure 15: Confusion matrices for the logistic regression (left) and decision tree (right) baselines when features with over 90% correlation were removed from the dataset.

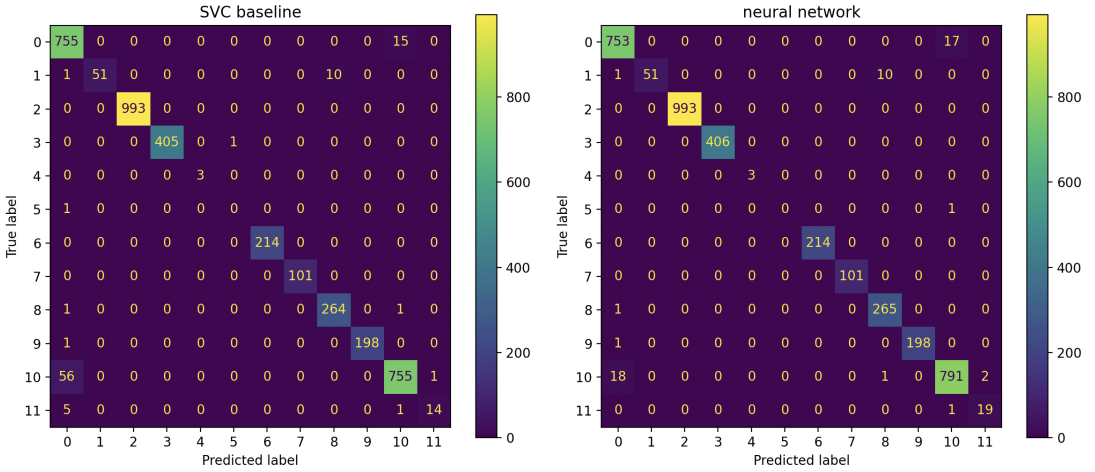

Figure 16: Confusion matrices for the SVC (left) and neural network (right) baselines when all data given was used..

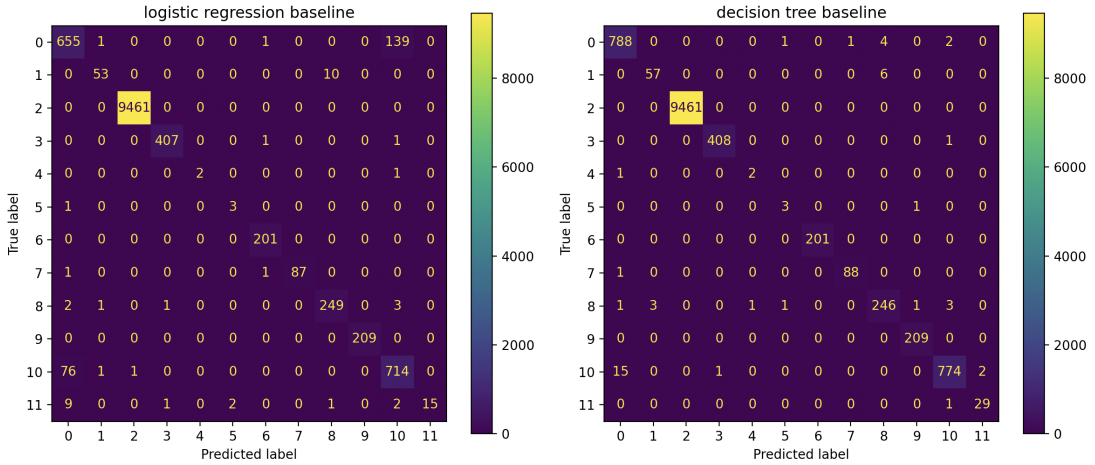

Figure 17: Confusion matrices for the logistic regression (left) and decision tree (right) baselines when data was reduced to 10,000 instances of dos_syn_hping.

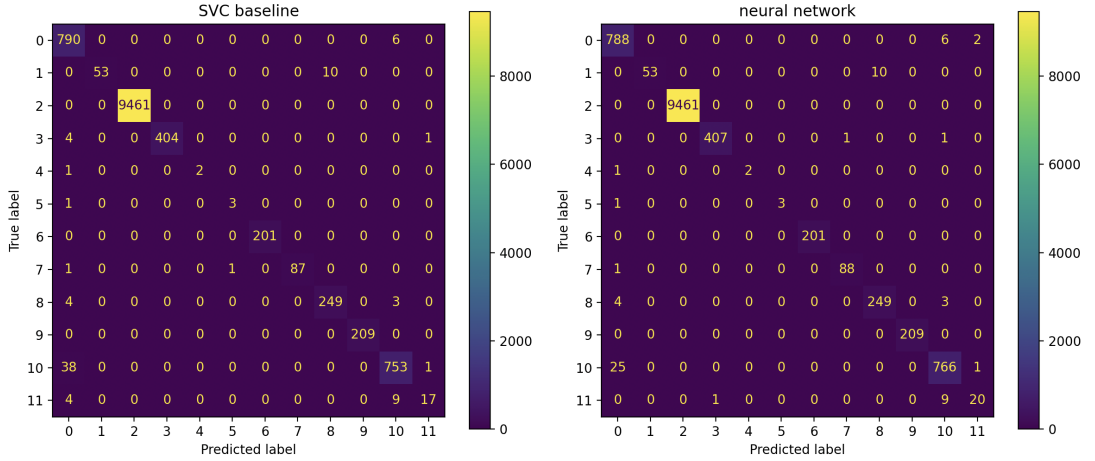

Figure 18: Confusion matrices for the SVC (left) and neural network (right) baselines when all data was reduced to 10,000 instances of dos_syn_hping.

Table 6: Zeek Description of Features

Feature Name	Description
flow_duration	The length of the flow in seconds (maximal precision ms). If only on packet was seen the duration is 0.
fwd_pkts_tot	The number of packets travelling in the forward direction.
bwd_pkts_tot	The number of packets travelling in the backwards direction.
fwd_data_pkts_tot	The number of packets travelling in the forward direction, which have a payload.
bwd_data_pkts_tot	The number of packets travelling in the backwards direction, which have a payload.
fwd_pkts_per_sec	The average number of forward packets transmitted per second during the flow. If the duration is 0 then this feature is also set to 0.
bwd_pkts_per_sec	The average number of backward packets transmitted per second during the flow. If the duration is 0 then this feature is also set to 0.
flow_pkts_per_sec	The average number of packets transmitted per second during the flow. If the duration is 0 then this feature is also set to 0.
down_up_ratio	The number of backward packets divided by the number of forward packets. If the number of forward packets is 0 this feature is also set to 0
fwd_header_size_tot	The total number of bytes the headers of the forward packets contained.
fwd_header_size_min	The number of bytes the smallest headers of the forward packets contained.
fwd_header_size_max	The number of bytes the largest headers of the forward packets contained.
bwd_header_size_tot	The total number of bytes the headers of the backward packets contained.
bwd_header_size_min	The number of bytes the smallest headers of the backward packets contained.
bwd_header_size_max	The number of bytes the largest headers of the backward packets contained.
fwd_pkts_payload.max	The largest payload size, in bytes, seen in the forward direction.
fwd_pkts_payload.min	The smallest payload size, in bytes, seen in the forward direction.
fwd_pkts_payload.tot	The total payload size, in bytes, seen in the forward direction.
fwd_pkts_payload.avg	The average payload size, in bytes, seen in the forward direction.
fwd_pkts_payload.std	The standard deviation of the payload size, in bytes, seen in the forward direction.
bwd_pkts_payload.max	The largest payload size, in bytes, seen in the backward direction.
bwd_pkts_payload.min	The smallest payload size, in bytes, seen in the backward direction.
bwd_pkts_payload.tot	The total payload size, in bytes, seen in the backward direction.
bwd_pkts_payload.avg	The average payload size, in bytes, seen in the backward direction.
bwd_pkts_payload.std	The standard deviation of the payload size, in bytes, seen in the backward direction.
flow_pkts_payload.max	The largest payload size, in bytes, seen in the flow.
flow_pkts_payload.min	The smallest payload size, in bytes, seen in the flow.
flow_pkts_payload.tot	The total payload size, in bytes, seen in the flow.
flow_pkts_payload.avg	The average payload size, in bytes, seen in the flow.
flow_pkts_payload.std	The standard deviation of the payload size, in bytes, seen in the flow
payload_bytes_per_second	The average number of payload bytes transmitted per second. If the duration is 0 then this feature is also set to 0.

Feature Name	Description
flow_FIN_flag_count	The total number of FIN flags which have been seen in a TCP flow. If the the flow is not a TCP flow this feature is set to 0.
flow_SYN_flag_count	The total number of SYN flags which have been seen in a TCP flow. If the the flow is not a TCP flow this feature is set to 0.
flow_RST_flag_count	The total number of RST flags which have been seen in a TCP flow. If the the flow is not a TCP flow this feature is set to 0.
fwd_PSH_flag_count	The total number of PSH flags which have been seen in the forward direction of a TCP flow. If the the flow is not a TCP flow this feature is set to 0.
bwd_PSH_flag_count	The total number of PSH flags which have been seen in the backward direction of a TCP flow. If the the flow is not a TCP flow this feature is set to 0.
flow_ACK_flag_count	The total number of ACK flags which have been seen in a TCP flow. If the the flow is not a TCP flow this feature is set to 0.
fwd_URG_flag_count	The total number of URG flags which have been seen in the forward direction of a TCP flow. If the the flow is not a TCP flow this feature is set to 0.
bwd_URG_flag_count	The total number of URG flags which have been seen in the backward direction of a TCP flow. If the the flow is not a TCP flow this feature is set to 0.
flow_CWR_flag_count	The total number of CWR flags which have been seen in a TCP flow. If the the flow is not a TCP flow this feature is set to 0.
flow_ECE_flag_count	The total number of ECE flags which have been seen in a TCP flow. If the the flow is not a TCP flow this feature is set to 0.
fwd_iat.max	The largest inter-arrival time in microseconds bet two consecutive packets in the forward direction.
fwd_iat.min	The smallest inter-arrival time in microseconds bet two consecutive packets in the forward direction.
fwd_iat.tot	The inter-arrival time in microseconds bet two consecutive packets in the forward direction.
fwd_iat.avg	The average inter-arrival time in microseconds bet two consecutive packets in the forward direction.
fwd_iat.std	The standard deviation of all inter-arrival times in the forward direction in microseconds.
bwd_iat.max	The largest inter-arrival time in microseconds bet two consecutive packets in the backward direction.
bwd_iat.min	The smallest inter-arrival time in microseconds bet two consecutive packets in the backward direction.
bwd_iat.tot	The inter-arrival time in microseconds bet two consecutive packets in the backward direction.
bwd_iat.avg	The average inter-arrival time in microseconds bet two consecutive packets in the backward direction.
bwd_iat.std	The standard deviation of all inter-arrival times in the backward direction in microseconds.
flow_iat.max	The largest inter-arrival time in microseconds bet two consecutive packets in the flow.
flow_iat.min	The smallest inter-arrival time in microseconds bet two consecutive packets in the flow.
flow_iat.tot	The inter-arrival time in microseconds bet two consecutive packets in the flow.
flow_iat.avg	The average inter-arrival time in microseconds bet two consecutive packets in the flow.
flow_iat.std	The standard deviation of all inter-arrival times in the flow, in microseconds.
fwd_subflow_pkts	The average number of packets in the subflows in the forward direction.
bwd_subflow_pkts	The average number of packets in the subflows in the backward direction.
fwd_subflow_bytes	The average number of payload bytes in the subflows in the forward direction.
bwd_subflow_bytes	The average number of payload bytes in the subflows in the backward direction.
fwd_bulk_bytes	The average number of payload bytes transmitted in a bulk transmission in forward direction.

Feature Name	Description
bwd_bulk_bytes	The average number of payload bytes transmitted in a bulk transmission in backward direction.
fwd_bulk_packets	The average number of packets transmitted in a bulk transmission in forward direction.
bwd_bulk_packets	The average number of packets transmitted in a bulk transmission in backward direction.
fwd_bulk_rate	The average number of payload bytes transmitted per second during a bulk transmission in forward direction.
bwd_bulk_rate	The average number of payload bytes transmitted per second during a bulk transmission in backward direction.
active.max	The longest duration the flow was active in microseconds.
active.min	The shortest duration the flow was active in microseconds.
active.tot	The total duration the flow was active in microseconds.
active.avg	The average duration the flow was active in microseconds.
active.std	The standard deviation of all active periods in microseconds.
idle.max	The longest duration the flow was idle in microseconds.
idle.min	The shortest duration the flow was idle in microseconds.
idle.tot	The total duration the flow was idle in microseconds.
idle.avg	The average duration the flow was idle in microseconds.
idle.std	The standard deviation of all idle periods in microseconds.
fwd_init_window_size	The window size in bytes the first packet in the forward direction has. The windows scale parameter is currently ignored, as this is only set in a SYN packet but we currently look at any packet
bwd_init_window_size	The window size in bytes the first packet in the backward direction has. The windows scale parameter is currently ignored, as this is only set in a SYN packet but we currently look at any packet.
fwd_last_window_size	The window size in bytes the last packet in the forward direction has. The windows scale parameter is currently ignored, as this is only set in a SYN packet but we currently look at any packet.
bwd_last_window_size	The window size in bytes the last packet in the backward direction has. The windows scale parameter is currently ignored, as this is only set in a SYN packet but we currently look at any packet.